Lecture 12

Data Structure

What is data Structure?

- A data structure is a method of storing and organizing data so that it can be accessed and modified efficiently.
- No single data structure works well for all purposes. It is important to understand the strengths and weaknesses of various data structures to choose appropriate one for given task.
- Data objects merely refers to set of elements, a data structure describes both the set of objects and specific way in which they are related to each other.

Need of Data Structure

- **Data Structure enables efficients algorithms:**

  For example, if you need to find a person's phone number in a massive list, a basic algorithm might have to examine every single name one by one, which is highly impractical. However, if that data is structured in alphabetical order, you can use a completely different, vastly more efficient searching algorithm.

- **Data Structure manages bounded resources:**

  Computer memory and processing time are not infinitely fast or free. Choosing the most efficient data structure helps conserve these limited resources by reducing the time and space required to execute a program

Applications of Data Structures

- **Stack**

  Processing subroutines call and their returns, back tracking -> finding a path through maze and evaluating mathematical expression

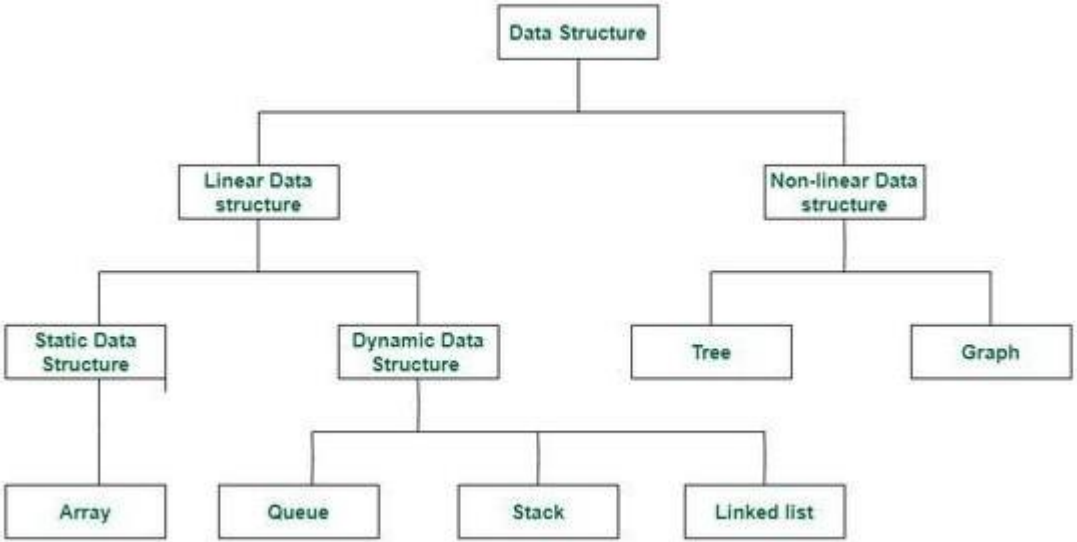
- **Queue**

  Queues are commonly used for scheduling jobs in batch processing systems or on shared computers.

- **Linked List**

  Linked lists are highly flexible and are used to represent and manipulate symbolic polynomials. They are also extensively used in operating systems for dynamic storage management.

Classification of Data Structure



Abstract Data Type

- Abstract Data Type (ADT) is a conceptual model. They define what a data structure does without dictating how it does it. By separating the logical properties from the physical implementation, ADTs allow for cleaner, more maintainable code.

  Example -> when a driver drives a car it and pushes break. He/she does not bother about the break system is working inside the car. He/she only want when i pull brake the car should stop.

ADT Array Structure

```
#include <stdio.h>

#include <stdlib.h>


// ADT Array Structure

typedef struct {

    int size;     // size of array

    int *data;    // pointer to elements

} Array;
```

Create Operation

```
Array createArray(int size) {
    Array A;
    A.size = size;
    A.data = (int*) malloc(size * sizeof(int));
    if (A.data == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }
    // initialize with undefined (0 for simplicity)
    for (int i = 0; i < size; i++) {
        A.data[i] = 0;
    }
   return A;
```

Retrieve Operation

```
void arrayStore(Array *A, int index, int value) {
    if (index >= 0 && index < A->size) {
        A->data[index] = value;
    } else {
        printf("Error: Invalid index!\n");
        exit(1);
    }
}
```

Main function

```
int main() {
    Array A = createArray(5);   // Create array of size 5
    arrayStore(&A, 0, 10);
    arrayStore(&A, 1, 20);
    arrayStore(&A, 2, 30);
    printf("Array elements:\n");
    display(A);
    printf("Element at index 1: %d\n", itemRetrieve(A, 1));
    free(A.data);  // Free memory

    return 0;
}
```

Store Operation

```
void arrayStore(Array *A, int index, int value) {
    if (index >= 0 && index < A->size) {
        A->data[index] = value;
    } else {
        printf("Error: Invalid index!\n");
        exit(1);
    }
}
```

Display Function

```
void display(Array A) {
    for (int i = 0; i < A.size; i++) {
        printf("%d ", A.data[i]);
    }
    printf("\n");
}
```